API Pagination

What is Pagination?

Pagination is the practice of **splitting large datasets into smaller, discrete pages** and returning one page at a time from an API, rather than returning all records in a single response.

Why Pagination Matters

Without pagination, fetching large datasets causes two major problems:

1. **Database strain** — querying all records puts heavy load on the database
2. **Slow responses** — sending massive payloads over the wire to the client degrades API performance significantly

> Real-World Analogy Think of Google Search results. How often do you visit page 2? Maybe 10% of the time. Page 3? Even less. This means the vast majority of records never need to be fetched at all.

The Core Tradeoff

Smaller page size	Larger page size
Less data per response	More data per response
More requests to the server	Fewer requests to the server
Faster individual responses	Slower individual responses

> ⚠ Design Consideration
>
> The ideal page size is **not universal** — it varies by resource type and use case. Some endpoints may justify a custom `/all` endpoint if the client reliably needs the full dataset.

API Response Structure

A well-designed paginated API **nests data** inside a `data` attribute, leaving room for pagination metadata alongside it.

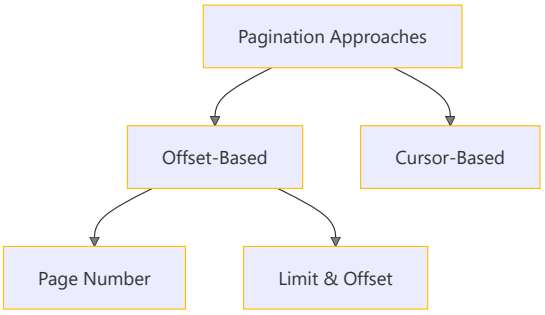
```
{
  "data": [ ...records... ],
  "count": 10000,
  "next": "https://api.example.com/comments?page=4",
  "previous": "https://api.example.com/comments?page=2"
}
```

Why Nest in `data`?

- Allows clean separation of **records** vs **pagination metadata**
- Clients know exactly where to look: `response.data` for records, `response.next` to navigate
- Makes it easy to add new metadata fields without breaking existing consumers

---

Types of Pagination



Offset-Based Pagination

Uses a **row/document offset** from the start of the dataset to determine where a page begins.

1a. Page Number Approach

The client provides a `page` query parameter. The server holds a default `page_size` (e.g. 20).

```
GET /comments?page=3
```

Page	Row Range
1	1 – 20
2	21 – 40
3	41 – 60

Offset Formula `offset = (page - 1) × page_size` For page 3: `(3 - 1) × 20 = 40`

Navigation attributes:

```
{
  "next": "/comments?page=4",
  "previous": "/comments?page=2"
}
```

SQL

```sql
SELECT * FROM comments
ORDER BY id
LIMIT 20
OFFSET 40;   -- (page - 1) × page_size
```

MongoDB (Beanie)

```python
from beanie import Document
from datetime import datetime
from typing import List

class Comment(Document):
    content: str
    created_at: datetime

    class Settings:
        name = "comments"

async def get_comments_page(page: int, page_size: int = 20) -> List[Comment]:
    offset = (page - 1) * page_size
    return await Comment.find() \
        .sort(Comment.id) \
        .skip(offset) \
        .limit(page_size) \
        .to_list()
```

Beanie Notes

- `.skip()` maps directly to MongoDB's `skip` cursor method
- `.sort(Comment.id)` uses the document field directly — Beanie resolves it to `"_id"`
- Always call `.to_list()` to execute and return results as a Python list

---

1b. Limit & Offset Approach

Exposes `limit` and `offset` directly as query parameters, giving the client more control.

```
GET /comments?limit=30&offset=120
```

Navigation:

```
{
  "next": "/comments?limit=30&offset=150",
  "previous": "/comments?limit=30&offset=90"
}
```

Client Control

This approach allows features like a **"results per page" dropdown** (e.g. 20 / 50 / 100). Always enforce a **maximum limit** server-side to prevent clients from fetching everything at once.

SQL

```sql
SELECT * FROM comments
ORDER BY id
LIMIT 30
OFFSET 120;
```

MongoDB (Beanie)

```python
async def get_comments(limit: int, offset: int) -> List[Comment]:
    # Enforce a max limit server-side
    limit = min(limit, 150)
    return await Comment.find() \
        .sort(Comment.id) \
        .skip(offset) \
        .limit(limit) \
        .to_list()
```

Getting the Total Count Use `.count()` separately to populate the `count` field in your API response:

```python
total = await Comment.find().count()
```

The Critical Weakness of Offset-Based Pagination

Offset pagination is **position-based** — it has no knowledge of actual record identity, only positional counts in the result set.

Problem: If data is **inserted or deleted** while a user is paginating, the offset shifts, causing:

- **Skipped records** — items the user never sees
- **Duplicate records** — same item appearing on multiple pages

Example:

```
Original:  [1, 2, 3, 4, 5, 6, 7, 8, 9, 10]
User sees page 1 (offset 0): [1, 2, 3]

Item 4 is deleted → dataset becomes:
           [1, 2, 5, 6, 7, 8, 9, 10]

User fetches page 2 (offset 3): [6, 7, 8]
→ Item 5 was SKIPPED entirely
```

Impact This is a disruptive experience — especially in real-time feeds or frequently updated datasets.

MongoDB `.skip()` Performance Warning Just like SQL `OFFSET`, MongoDB's `.skip()` must scan and discard all preceding documents. On large collections this degrades significantly with depth. For collections with millions of documents, prefer cursor-based pagination.

Cursor-Based Pagination

Uses a **pointer to the last record seen** rather than a positional offset. The database starts from that specific record, making it immune to inserts and deletes earlier in the dataset.

How it Works

Instead of "skip N documents," the query says **"give me records after this specific ID."**

Transparent Cursor (Avoid in Production)

```
GET /comments?after_id=12378
```

⚠ Anti-Pattern

Exposing raw IDs leaks database structure, limits flexibility, and creates directional confusion (`after` implies forward-only). Avoid this in public APIs.

Continuation Token (Recommended)

The cursor is **encoded into an opaque token**, hiding implementation details:

```
GET /comments?continuation=eyJpZCI6MTIzNzh9...
```

```
{
  "data": [ ...records... ],
  "next": "/comments?continuation=eyJpZCI6MTIzNzh9..."
}
```

Benefits of Continuation Tokens

- Hides database internals (especially useful in MongoDB where `_id` is an ObjectId)
- Allows backend implementation to change without breaking the client contract
- The client just passes the token — no calculation needed

SQL

```sql
SELECT * FROM comments
WHERE id > 12378
ORDER BY id
LIMIT 20;
```

MongoDB (Beanie) — Cursor by `_id`

MongoDB's `ObjectId` is **ideal** for cursor-based pagination — it is unique, immutable, indexed by default, and encodes a timestamp, making it naturally sortable by insertion order.

```python
from bson import ObjectId
import base64, json

async def get_comments_cursor(
    limit: int = 20,
    after: str | None = None   # continuation token
) -> dict:

    # Decode the continuation token back to an ObjectId
    query = {}
    if after:
        last_id = ObjectId(json.loads(base64.b64decode(after))["id"])
        query = {Comment.id: {"$gt": last_id}}

    results = await Comment.find(query) \
        .sort(Comment.id) \
        .limit(limit) \
        .to_list()

    # Encode the last seen ID as a continuation token
    next_token = None
    if len(results) == limit:
        payload = json.dumps({"id": str(results[-1].id)})
        next_token = base64.b64encode(payload.encode()).decode()

    return {
        "data": results,
        "next": f"/comments?continuation={next_token}" if next_token else None
    }
```

Why ObjectId Works So Well

MongoDB's `ObjectId` is a 12-byte value that includes a **4-byte timestamp**, meaning documents are already insertion-ordered by `_id` out of the box. No extra `created_at` index is needed for basic cursor pagination.

MongoDB (Beanie) — Cursor by `created_at`

If you need to sort by time explicitly (e.g. for feeds where IDs may not reflect insertion order):

```python
from datetime import datetime

async def get_comments_by_time(
    limit: int = 20,
    after: str | None = None
) -> dict:

    query = {}
    if after:
        decoded = json.loads(base64.b64decode(after))
        last_time = datetime.fromisoformat(decoded["created_at"])
        query = {Comment.created_at: {"$gt": last_time}}

    results = await Comment.find(query) \
        .sort(Comment.created_at) \
        .limit(limit) \
        .to_list()

    next_token = None
    if len(results) == limit:
        payload = json.dumps({"created_at": results[-1].created_at.isoformat()})
        next_token = base64.b64encode(payload.encode()).decode()

    return {
        "data": results,
        "next": f"/comments?continuation={next_token}" if next_token else None
    }
```

⚠ **Timestamp Collision Risk** If multiple documents can share the exact same `created_at` value, using it alone as a cursor can skip records. For high-throughput collections, combine `created_at` + `_id` in the cursor token to guarantee uniqueness.

Cursor-Based: Requirements for the Cursor Field

Property	Reason
Unique	Unambiguously identifies a record
Not null	Cannot be missing
Unchanging	Can't shift after creation
Indexed	Enables efficient lookups

Field	SQL	MongoDB / Beanie
✅ Best choice	`id` (auto-increment)	`_id` (ObjectId)
✅ Good alternative	`created_at`	`created_at` (with index)
❌ Avoid	`name`, `status`, `price`	same — mutable / non-unique

Declaring Indexes in Beanie Always index your cursor field to keep lookups fast:

```python
class Comment(Document):
    created_at: datetime

    class Settings:
        name = "comments"
        indexes = ["created_at"]  # ensures fast cursor lookups
```

Offset vs Cursor — Head-to-Head Comparison

Feature	Offset-Based	Cursor-Based
Arbitrary page jumps	✅ Yes	❌ No
Arbitrary sort/filter	✅ Yes	❌ Limited
Immune to data changes	❌ No	✅ Yes
Ideal for large datasets	❌ Degrades with depth	✅ Consistent performance
Client UX	Numbered pages	Load more / Infinite scroll
Implementation complexity	Simple	Moderate
SQL equivalent	`LIMIT n OFFSET skip`	`WHERE id > x LIMIT n`
MongoDB / Beanie equivalent	`.skip(offset).limit(n)`	`.find({"_id": {"$gt": ObjectId(x)}}).limit(n)`

> The core reason offset-based pagination degrades with depth is that **the database must scan and discard all previous rows for every query**.

When you run `SELECT * FROM users ORDER BY id LIMIT 10 OFFSET 100000`, the database doesn't magically jump to row 100,001. It must:

1. **Find** the first 100,010 rows that match your `WHERE` and `ORDER BY` clauses.
2. **Sort** them (if an index isn't a perfect match).
3. **Count** out the first 100,000 rows.
4. **Discard** them.
5. **Return** the remaining 10.

As `OFFSET` grows, the work grows **linearly** with the offset value.

Why Indexes Don't Fully Solve It

Even with an index on the `ORDER BY` column:

- **Index scan still starts at the beginning** – The database still has to traverse the index from the first entry until it has skipped `OFFSET` rows.
- **Each page is not independent** – Page 10,001 requires re-scanning the same first 10,000 rows that page 1 already scanned.

Choosing the Right Client-Side Pattern

Use Offset-Based When:

- Users need to **jump to a specific page** (e.g. "Go to page 36")
- Data needs **arbitrary sorting and filtering** (e.g. filter by make, model, price)
- A classic **numbered pagination UI** is expected

Use Cursor-Based When:

- Implementing **infinite scroll** or **"Load More"** UX
- Working with **real-time or frequently updated feeds**
- Handling **very large datasets** where query performance matters
- Data integrity during navigation is critical (no skips or duplicates)

Handling Duplicates in Offset + Load More

If you combine **offset-based pagination** with an **append-to-state** (load more) pattern, you must **deduplicate** on the client side.

Scenario:

```
State: [Comment A, Comment B, Comment C]
→ New comment inserted before offset point
→ Next fetch (offset 3) returns: [Comment C, Comment D, Comment E]
→ Comment C is now duplicated in local state!
```

Solution Deduplicate by ID before appending new records to local state. This is not needed with cursor-based pagination — it handles this seamlessly by design.

Quick Reference Cheatsheet

```
Offset Formula:   offset = (page − 1) × page_size
Next URL:         offset + limit
Previous URL:     offset − limit
```

Operation	SQL	MongoDB / Beanie
Page N (offset)	`LIMIT n OFFSET skip`	`.skip(offset).limit(n)`
Cursor fetch	`WHERE id > x LIMIT n`	`.find({"_id": {"$gt": ObjectId(x)}}).limit(n)`
Total count	`SELECT COUNT(*) FROM …`	`await Model.find().count()`
Sort ascending	`ORDER BY id ASC`	`.sort(Model.id)`
Sort descending	`ORDER BY id DESC`	`.sort(-Model.id)`

```
Continuation Token:
  Encode:  base64( json({"id": str(last_doc.id)}) )
  Decode:  ObjectId( json(base64decode(token))["id"] )
  Always set next: null when results < limit  →  signals last page
```

Key Takeaway Offset-based = simple, flexible, supports page jumping — but degrades with dataset size and is vulnerable to data-change artifacts. In MongoDB/Beanie, powered by `.skip()`. Cursor-based = resilient, performant at scale — but limits navigation to sequential access. In MongoDB/Beanie, powered by range queries on `_id` (ObjectId). MongoDB's ObjectId is a natural fit — unique, immutable, indexed, and timestamp-encoded by design. Match your pagination strategy to your data characteristics and client UX requirements.